From Graphs to Predictions: a Tiling Problem Resolution

Adam Azouz
University of Lausanne
adam.azouz@unil.ch

***Abstract*—The project addresses the original and challenging algorithmic problem of determining the feasibility of tiling a given layout with 2:1 tiles. Our objective is to resolve this problem optimally, study its complexity, and extend its scope by exploring more intriguing questions that bridge concepts from machine learning and the law of large numbers. We start by considering layouts, such as rooms or gardens, with predefined dimensions and obstacles scattered throughout. Using sophisticated concepts, we take on the task of determining whether a given layout is tilable. Expanding our investigation, we introduce a probabilistic dimension to the problem by assigning probabilities to the existence of obstacles or walls within each cell or line of the layout. This augmentation allows us to assess the likelihood of the layout being tilable under stochastic obstacle placement scenarios. In our exploration of the intersection between algorithmic problem-solving and machine learning, we investigate the potential applications of machine learning techniques to enhance our solution to the tiling problem. Leveraging predictive analytics and insights derived from data, our objective is to study the benifits machine learning can bring for efficiently determining layout tilability when assigning a probability of obstacle existence. Through this interdisciplinary problem, we try not only to provide practical solutions to the proposed problem but also to push the boundaries of algorithm solving by incorporating methodologies from machine learning.**

***Index Terms*—Algorithms, Machine Learning, Graph Theory, Ford-Fulkerson algorithm, Probability, Polynomial complexity**

I. Introduction

Algorithms serve as the cornerstone of computational problem-solving, providing systematic and efficient procedures for tackling a wide range of tasks in various domains. From performing simple arithmetic operations to solving complex optimization problems, algorithms play a pivotal role in modern society, powering everything from search engines and social media platforms to financial systems.

At its core, an algorithm is a well-defined set of instructions or rules that specify a sequence of steps for performing a particular task or solving a problem and must exhibit the following key characteristics: 1.Correctness: An algorithm must produce the correct output for all valid inputs, adhering to the problem's specifications and constraints. 2.Efficiency: An algorithm should execute within a reasonable amount of time and utilize a minimal amount of computational resources (e.g., memory, processing power). 3.Generality: An algorithm should be applicable to a broad range of inputs and problem instances, demonstrating versatility and scalability across different scenarios.

In our project, we leverage the power of algorithms, with a particular emphasis on graph theory, to address the intricate challenges posed by the tiling problem with obstacles. Graph theory serves as the backbone of our project, and is a fundamental toolset in our proposed solution.

Graph theory is fundamentally crucial in the field of algorithms, where it acts as a key framework for modeling, analyzing, and addressing various computational challenges. Its importance is derived from its capability to succinctly and clearly depict intricate relationships and frameworks, thereby becoming a vital collection of tools for the creation and examination of algorithms.

Essentially, graph theory focuses on the analysis of graphs; these are mathematical constructs made up of vertices (or nodes) and edges (or links) that connect pairs of vertices. Moreover, graph theory provides a rich array of concepts, algorithms, and results that form the basis for many fundamental algorithms and data structures. Graph algorithms, such as breadth-first search (BFS), depth-first search (DFS), Dijkstra's algorithm, and the Ford-Fulkerson algorithm, are among the most widely used algorithms in computer science.

In this project, we employ the Ford-Fulkerson algorithm in conjunction with BFS to effectively traverse the network. By utilizing the principles of residual networks, we make use of flow maximization, to boost the performance of the algorithm. This method allows us to address the proposed problem in a optimized way, with the goal of achieving great efficiency.

Machine learning constitutes the last dimension of our project, offering a powerful approach to push even further the efficiency of our algorithmic solution. By harnessing machine learning techniques, we aim to enhance the predictive capabilities of our model, enabling more time efficient decision-making in complex scenarios (high layout dimensions). This integration of machine learning extends the effectiveness of our solution, and provides a good overview of benefits we can get out of such techniques.

We believe that achieving a close approximation of the actual outcome from a time-intensive algorithm in a much shorter duration holds considerable significance, as one can decide with more information at hand, whether to allocate the necessary resources for the exact calculations. And we try to showcase this in our project.

In addition to the algorithmic and machine learning components, we have incorporated a Command-Line Interface (CLI) into our project to interact with the underlying algorithm

and model. The CLI serves as a user-friendly interface that allows users to input parameters, and execute the algorithm with ease. Through the CLI, users can interactively engage with the functionalities of our project, providing for example customized layouts.

II. Research Question

A. Problem

Given a n*m layout of a space (garden, appartement, ...) where there is non-tillable areas(walls, constructions, ...) and given rectangular tiles of length to width ratio of 2:1.

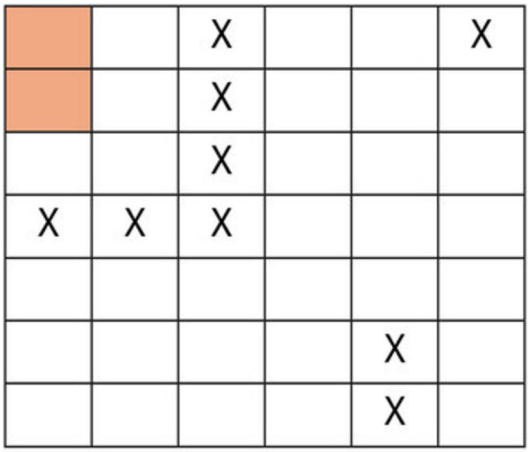


Fig. 1. Example of a layout. Cells with the X symbol represent non-tillable areas. The colored cells are an example to represent the area covered by a single tile.

We want to determine whether it is possible to tile the layout, where every tile covers exactly two horizontally or vertically adjadjacent spaces of the layout. Every space that is to be tiled must be covered by exactly one tile while the other spaces must remain untiled. The supply of tiles is unlimited. In the example given by Fig.1, the space can't be tiled according to our specification.

Once we successfully resolve the core algorithmic problem at hand, our exploration does not cease; instead, we delve deeper by exploring related and intriguing questions. Using the algorithmic solution, we expand on further exploration of the problem space by introducing a probabilistic dimension.

Specifically, we introduce a probability p denoting the likelihood of obstacle existence for each cell within the layout. This addition allows us to probe deeper into the probabilistic nature of the tiling problem, posing intriguing questions such as: given the dimensions n*m of the layout and the probability p of obstacle existence, what is the probability that the room is tilable? From this we investigate interesting aspects such as the relation between the layout dimensions and the layout tilability. Furthermore, we also analyze the impact of varying obstacle existence probabilities p on the tilability probability, unraveling insights into how different levels of obstruction can affect the feasibility of tiling. Through this probabilistic lens, we try to discover some hidden relationships and patterns.

We additionally consider a parallel question, where this time we consider the probability p of wall existence. A wall, in this context, is an obstacle spanning over an entire line or column within the layout. We again try and analyze the complex relationships that exist between the dimensions of layout, the probability p of wall existence and the probability of tilability of the layout.

After considering the high computational cost of answering these questions, we explored more cost-effective methods for generating accurate approximations.

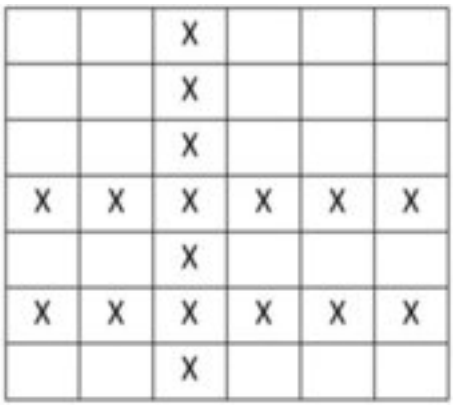


Fig. 2. Example of wall obstacles.

This led us to explore predictive technologies that could provide efficient solutions without sacrificing precision. At this juncture, machine learning emerged as a promising technique to address our computational challenges. Using the Kernel-ridge regression technique, We sought to develop predictive models capable of estimating the probability of room tilability based on layout dimensions and obstacle probabilities. By training these models on generated datasets (using the previously developed algorithm), we aimed to get a good sense of the possibilities that this technology can offer.

B. Graph Theory: An overview

Graph theory is a branch of mathematics that deals with the study of graphs. A graph is a mathematical structure that consists of vertices (also called nodes) and edges that connect pairs of vertices. Graphs are widely used to model relationships between objects in various fields, including computer science, engineering, social sciences, and biology.

Basic Definitions:

- Graph: A graph G is defined as a pair (V, E), where V is a set of vertices and E is a set of edges. Each edge in E connects two vertices in V.
- Vertex: A vertex, or node, is a fundamental element of a graph. It represents an entity or object in a given context. Vertices are typically denoted by letters or numbers.
- Edge: An edge is a connection between two vertices in a graph. It represents a relationship or interaction between the corresponding entities. Edges can be directed or undirected, depending on whether the relationship they represent has a

specific direction.
- Directed graph: In a directed graph, each edge has a direction associated with it.
- Flow graph (network): is a directed graph where each edge has a capacity and each edge receives a flow. The amount of flow on an edge cannot exceed the capacity of the edge.

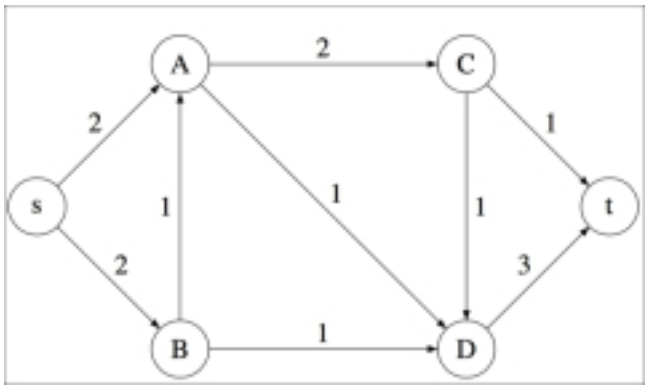


Fig. 3. Example of a flow graph. Vertex s being the source, and t the sink.

C. Ford-Fulkerson algorithm

The Ford-Fulkerson algorithm is a popular method for computing the maximum flow in a flow network. It operates by iteratively increasing the flow along paths from the source to the sink until there are no more augmenting routes. Here's how the algorithm works:

- Initialization:
Start with an initial flow of zero throughout the network. Initialize a residual graph that represents the remaining capacity of each edge in the network.

- Find an Augmenting Path:
Use a graph traversal algorithm, such as Breadth-First Search (BFS) or Depth-First Search (DFS), to find an augmenting path from the source to the sink in the residual graph. An augmenting path is a simple path from the source to the sink along which additional flow can be sent.

- Determine the Augmentation Amount:
Determine the maximum amount of flow that can be sent along the augmenting path found in the previous step. This is the minimum residual capacity of the edges along the path.

- Update the Flow:
Update the flow along the augmenting path by adding the augmentation amount to each edge in the path. Subtract the amount of augmentation from the residual capacity of the forward edges and add it to the residual capacity of backward edges in the residual graph.

- Repeat Until No Augmenting Paths Exist:
Repeat steps 2-4 until no augmenting paths can be found in the residual graph. When there are no augmenting paths, the flow in the network is maximal and cannot be further increased.

- Compute the Maximum Flow:
Calculate the maximum flow in the network by summing up the flow values leaving the source vertex. This represents the total flow sent from the source to the sink.

- Output:
Output the maximum flow value and the flow distribution across the edges in the network.

As such, the Ford-Fulkerson algorithm provides us with an efficient method for computing the maximum flow in a flow network.

D. Machine learning: An overview

Machine learning is a branch of artificial intelligence (AI) that focuses on the development of algorithms and models that enable computers to learn from and make predictions or decisions based on data. Unlike traditional computer programs that rely on explicit instructions, machine learning algorithms learn patterns and relationships from data to autonomously improve their performance over time.
Key Concepts of Machine Learning:

- Data: Data are at the core of machine learning. Machine learning algorithms require access to a dataset that contains relevant information on the problem being addressed. This data is typically divided into features (input variables) and labels (output variables).
- Training: In the training phase, a machine learning model is presented with a dataset containing input-output pairs. The model learns from this data by adjusting its internal parameters to minimize the difference between its predictions and the true labels.
- Generalization: Once trained, a machine learning model can generalize its knowledge to make predictions on new, unseen data. The goal is to develop a model that can accurately predict outcomes for previously unseen instances.
- Supervised vs. Unsupervised Learning: In supervised learning, the model learns from labeled data, where each instance in the dataset is associated with a corresponding label. In unsupervised learning, the model learns patterns and structures from unlabeled data without explicit supervision.

Role of Machine Learning in Predictive Modeling:
Predictive modeling is the process of using historical data to make predictions about future outcomes. Machine learning plays a crucial role in predictive modeling by enabling the development of models that can learn patterns and relationships from data to make accurate predictions.
Applications of Predictive Modeling:
- Classification: Predictive models can be used for classification tasks, where the goal is to assign instances to predefined categories or classes. For example, email spam detection,

image classification, and disease diagnosis are common classification tasks.

- Regression: Predictive models can also be used for regression tasks, where the goal is to predict a continuous-valued output based on input features. Regression models are commonly used in financial forecasting, demand prediction, and housing price estimation.

- Anomaly detection: Predictive models can detect anomalies or outliers in data, which may indicate unusual behavior or events. Anomaly detection is used in fraud detection, network security, and predictive maintenance.

For our purposes, we are going to use a supervised learning model for our regression task.

E. Kernel ridge regression

Kernel ridge regression (KRR) is a powerful supervised learning algorithm used for regression tasks. It combines elements of both linear regression and kernel methods to model complex nonlinear relationships between input features and target variables.

1. Regression Problem:
As previously mentioned, the goal in regression problems is to predict a continuous-valued output (target variable) based on one or more input features (predictor variables). For example, prediction of the probability of tilability based on the layout dimensions and the probability of obstacle existence.

2. Ridge Regression:
Ridge regression is a linear regression technique that introduces a regularization term to the standard least squares objective function. The regularization term penalizes large coefficients, preventing overfitting and improving the generalization performance of the model.
Mathematically, ridge regression minimizes the following objective function: $\min_w \left\{\|Xw - y\|_2^2 + \alpha\|w\|_2^2\right\}$, where X is the matrix of input features, y is the vector of target variables, w is the vector of regression coefficients, and α is the regularization parameter.

3. Kernel Trick:
The kernel trick is a technique used to implicitly map input features into a higher-dimensional feature space without explicitly computing the transformed feature vectors. This allows linear algorithms, such as ridge regression, to model non-linear relationships between input features and target variables.
The kernel function $K(x_i, x_j)$ computes the inner product of the feature vectors in the transformed feature space without explicitly computing the transformation. Common kernel functions include polynomial kernels, radial basis function (RBF) kernels, and sigmoid kernels.

4. Kernel Ridge Regression:
Kernel ridge regression extends ridge regression by applying the kernel trick to map input features into a higher-dimensional feature space. Instead of solving the regression problem in the original feature space, KRR solves it in the kernel-induced feature space, where non-linear relationships can be captured more effectively. Mathematically, kernel ridge regression minimizes the following objective function: $\min_\alpha\{\|K\alpha - y\|_2^2 + \lambda\alpha^T K\alpha\}$ where K is the kernel matrix computed from the input features, α is the vector of dual coefficients, and λ is the regularization parameter.

5. Predictions:
Once the optimal coefficients α are obtained, predictions can be made for new instances by computing the inner product of their feature vectors with the support vectors in the kernel-induced feature space.

KRR can capture complex nonlinear patterns in the data and produce accurate predictions. Its regularization properties help prevent overfitting, making it suitable for a wide range of regression tasks, and we believe it is a good method to give us a preview of what machine learning techniques can offer in our application.

III. Methodology

A. The core algorithm

Our solution to the problem is based on graph theory, and particularly the use of the max-flow algorithm.
Given a n*m layout with obstacles, which we can see as a grid with tilable and non-tilable cells, we first check for the trivial case where the number of tiles to cover is odd. In this case we output that it is not possible to tile the layout. Next, in case the number is even, we transform the input into a flow graph as follows:
-Each cell in our grid becomes a vertex in our graph.
-We add a source and a sink vertex.
-We divide our grid, just like a chess board, into "white" and "black" cells (regardless of obstacle existence or not). This means that we get two sets of cells: one that satisfies $(i+j) = 0 mod 2$ and the other $(i+j) = 1 mod 2$. Where i and j are the coordinates of the cell in the grid.
-Now we connect the source vertex to each "black" vertex by a directed edge going from the source to the black cell with a capacity of 1. Except when there is an obstacle in the cell corresponding to the vertex.
-We connect each "white" vertex to the sink by a directed edge of capacity 1. Except when there is an obstacle in the cell corresponding to the vertex.
-Now we connect each black vertex to its neighboring white vertices with a directed edge of capacity 1. Except when there is an obstacle in the cell corresponding to the black vertex.
The rational behind dividing out grid in the way we did is to have every two adjacent cells be in different sets. so that we can next match them using a max-flow algorithm (as we are tiling with 2:1 tiles).
Now that we have created our graph, we run the Ford-Fulkerson algorithm on it. As an output, we get the maximum

flow that we can achieve in the graph.
The last step is to check whether the maximum-flow we got is equal to the total number of tiles to cover divided by two. If it is equal, then it is indeed possible to tile the layout. Otherwise, it is not possible.

The complexity of the Ford-Fulkerson algorithm is $O(E*f)$, where E is the number of edges in the graph, and f is the maximum flow of the graph. The creation of the graph itself from the input data can be performed in $O(E)$. Hence the complexity of our algorithm is $O(E * f)$. In our case, the number of edges is $n * m$, where n and m are the dimensions of the layout. The maximum flow f cannot exceed n*m/2 by construction (as only half the vertices are connected to the source). We can therefore conclude that the total complexity of our algorithm is $O((n * m)^2)$.

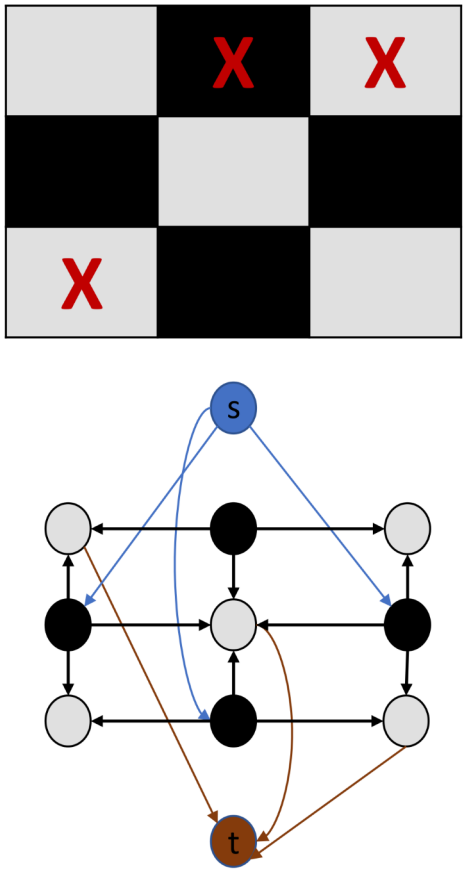


Fig. 4. Example of the transformation of a grid into a graph. The vertex s represents the source, while vertex t represents the sink. All the edges have capacity 1.

B. Estimating the probability of tilability

Given the dimensions $n * m$ of a layout and a probability p of obstacle existence in each cell, we want to determine how likely it is that the generated space is tilable. To do so, we rely on the law of large numbers.

The law of large numbers has a very central role in probability and statistics. It states that if you repeat an experiment independently a large number of times and average the result, what you obtain should be close to the expected value.
We generate a defined number N of different layouts of the given dimensions $n*m$. These generated layouts are randomly and independently generated, in the sense that a pseudorandom generator is used to determine, with the given probability p, whether an obstacle will be placed for each cell.
We then run the experiment, by running our algorithm on each of the N generated grids, and divide the total number of tilable instances by N. The result of the division is our estimation of the probability of tilability.

In the same way, we also estimate the probability of tilability for layouts for which we have placed, with a given probability p, horizontal and vertical walls.

C. Computational limitations

While generating some data to visualize the impacts of our parameters (n, m, p) on the probability of tilability, we encountered a massive running time bottleneck. To get a better feeling of the limitations we have, we consider the known value of 10^7 operations per second. We also consider N to be 100. This means that to estimate the probability of tilability of a $50 * 50$ layout, for a given probability of obstacle existence p, we need: $(50 * 50)^2 * 100 = 6.25 * 10^8$ operations, which translates to more than 1 minute. If we want to generate 100 different points, where we vary the probability p of obstacle existence, for this unique case of 50*50 layout then this would means almost 2 hours of running time.
That is why we considered experimenting with machine learning, as it can be a potential solution to our computational bottleneck.

D. Leveraging predictive methods

We made a basic machine learning model. Our main goal wasn't to create the most perfect or fastest model but to see how these prediction methods could help us. By keeping things simple, we aimed to understand how these techniques work and what they can offer us in our context.

To develop our Kernel Ridge Regression (KRR) model that predicts the probability of tilability based on the layout dimensions n and m, as well as the obstacle probability p, several steps and methodologies were employed:

- Data Collection: We generated a dataset consisting of 500 layout samples, each characterized by its dimensions n and m, along with the obstacle probability p and the corresponding estimation of the probability of tilability that we got using the previously described method.
- Data Preprocessing: We performed simple data preprocessing, starting with data normalization and then splitting the dataset into training and testing sets to evaluate the model's performance.
- Kernel Selection: We chose an appropriate kernel function for the KRR model, after experimenting with different kernel functions. The most simple and suitable one was the RBF kernel.

- Hyperparameter Tuning: Tune the hyperparameters of the KRR model, including the regularization parameter λ and . Use techniques such as cross-validation to find the optimal hyperparameter values that minimize prediction error.
- Model Training: Train the KRR model using the whole dataset.
- Model Deployment: Deploy the trained KRR model to be queried as part of our CLI application to predict the probability of tilability for new layout samples.

IV. Implementation

The project is hosted in a Github repository that contains all the required files. The program runs in Python, version 3.10 and is structured using the following packages:

- Tkinter - version 8.6.14
- Numpy - version 1.26.4
- Pandas - version 1.5.2
- Matplotlib - version 3.8.4
- Scikit-Learn - version 1.4.2
- Click - version 8.1.7

A. Structure

The most important files making up our project will be presented and are listed below:

```
code
    main.py
    models.py
    graph.py
    grid.py
    utility_funcs.py
    ...
inputs
    ...
README.md
```

First we create a ”Grid” data structure, by defining a Grid class in grid.py; as it is one of the most simple and natural ways to represent a layout. The defined class provides methods for accessing grid elements, dimensions, count of cells that must be tiled, adding obstacles to grid cells, add walls in rows or columns, and eliminating obstacles. Additionally, it has a method to print the grid. The class is designed to facilitate operations related to defining the space before proceeding to processing it into a graph.

Next, we define a Graph class in graph.py. We made the implementation choice of reprensenting the graph as an adjacency matrix, instead of an adjacency list. The class provides methods to add directed edges with capacities, get the number of vertices, perform a residual breadth-first search to find paths in the residual graph, and implement the Ford-Fulkerson algorithm to find the maximum flow from a source to a sink in the graph. This class is built to be particularly useful for modeling and analyzing flow problems.

In the file utility_funcs.py, are defined some of the core functions of our project. Most notably, a function which converts a grid to a flow graph as specified in details in the the previous section. Another, which, given a grid, makes use of all the defined modules and functions to output whether the layout is tilable or not. Functions to generate random grids, given the parameters (n,m,p), both for the cell obstacle case and the wall obstacle one. And finally, functions to estimate the probability of tilability for given parameters.

We define our machine learning model as part of the models.py file, where a class Model is defined. The class is initialized with a dataset and a standard scaler. The ”train” method fits the model to the data using Randomized Search Cross Validation to find optimal hyperparameters. The ”predict” method predicts the probability of tilability based on given layout dimensions n and m, and obstacle probability p.

Finally, in main.py is defined the code that utilizes all the functionalities and defines the code for our Command Line Interface (CLI). the CLI provides 3 main commands:

- istilabel n m : where n and m are integers. With this command a GUI pops up, and you can input the obstacles in your layout. The program then outputs for you whether the given layout is tilable or not.
- compute n m p c : where n and m are integers, p a float between 0 and 1, and c a char where ’w’ indicates obstacles of type wall. The program will then estimate, using the algorithm, what is the probability of tilability of the room.
- predict n m p c : where n and m are integers, p a float between 0 and 1, and c a char where ’w’ indicates obstacles of type wall. The program will then predict, using a machine learning model, what the probability of tilability of the room is.

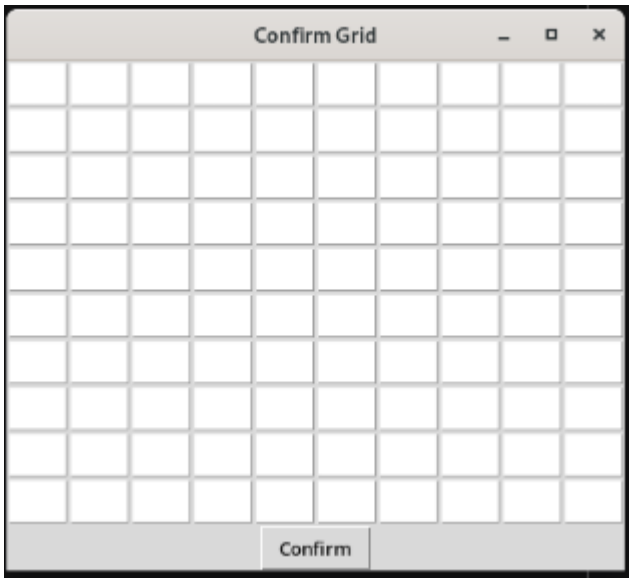


Fig. 5. Example of the GUI that pops when executing the ’istilabel’ command. In this example the exact excuted command is: ’istilabel 10 10’.

B. Testing the algorithm

The core algorithm has been tested with layouts that can be found in the inputs directory. The testing code is in 'test.py' file in the 'code' directory.

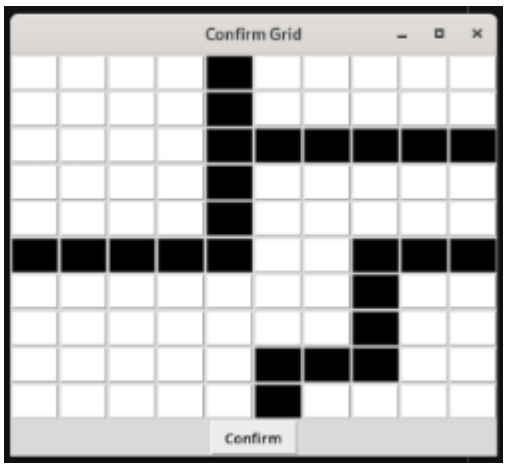


Fig. 6. The black colored cells represent the entered obstacles.The user inputs the obstacles in the GUI by clicking on the desired cells.

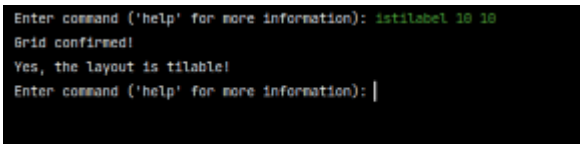


Fig. 7. Output of the CLI after clicking on the "confirm" button for the previous room layout.

V. Maintenance and Update

In our project, we maintain and update our code base using Git, a version control system that tracks changes and facilitates collaboration. Our code base is organized into modular components for readability and scalability. We utilize Git repositories hosted on the GitHub platform, which ensures accessibility and enables seamless collaboration among interested members. To maintain code quality, we provide documentation for onboarding and reference. By adhering to these practices, we ensure the long-term sustainability and effectiveness of our project.

VI. Results

A. The core algorithm

The core algorithm is effective, and can be used for specific cases through the 'istilabel' command in the CLI as shown previously. It can also be used in a more extensive way by making minor adjustments to the test.py file to enable processing larger number of inputs by reading from a provided input file. The first line of the input file should contain the number t of cases that are going to follow. Every case should then start with a line containing two space-separated integers, n and m, specifying the height and width of the layout. The next n lines should then describe one row of the layout. Each such line consists of m characters, either '.' if that space is to be tiles, or 'x' if not.

Example of an input file:

```
1       2
2       7 3
3       x.xxxxx
4       x.....
5       xxx.xx.
7       9 4
8       xxxxxxx.x
9       xxxxxxx.x
10      xxxxxxxxx
11      xxxx..xxx
```

B. Estimating the probability of tilability

1) Cell obstacles: First, we examine the tilability probability across various layout dimensions, with a probability p of obstacle existence. We specifically observe how the tilability probability fluctuates when altering one dimension with fixed p, and then altering the probability p with fixed dimensions. (fig.8 to fig.14).

2) wall obstacles: We next visualize the same graphics, with the same parameters, but for wall obstacles. it's important to mention that our focus is solely on low-dimensional layouts due to the significant increase in computational resources required for higher dimensions. It's worth noting that the generation of this data alone consumed tens of hours, but it could have been parallelized. (fig.15 to fig.21)

3) Machine learning: We finally present figures to visualize the predictions of our two machine learning models against the actual values. The machine learning approach shows promise and has the potential to provide good approximations. However, in our case, its efficiency is highly dependent on the range of input values, given that we only used 500 data points for training. Additionally, the model employed is quite simplistic, which further impacts its overall accuracy.

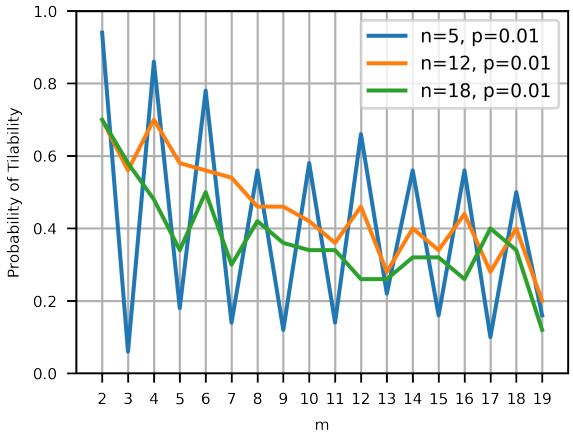


Fig. 8. Cell obstacles: Varying dimension, p=0.01.

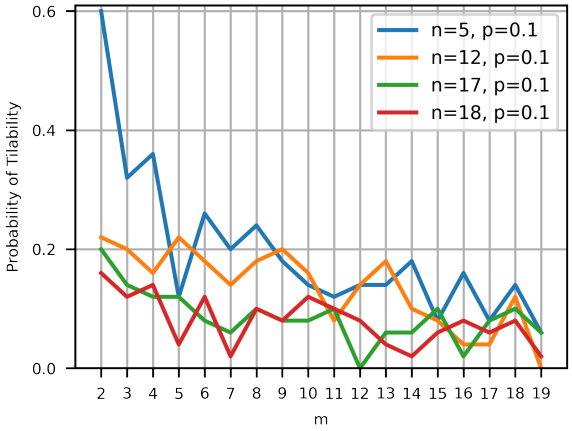


Fig. 9. Cell obstacles: Varying dimension, p=0.1.

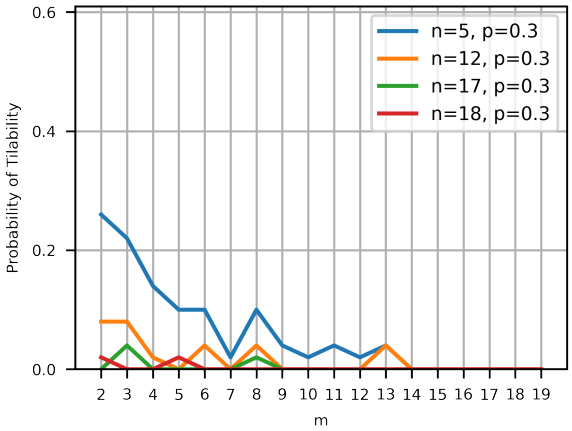


Fig. 10. Cell obstacles: Varying dimension, p=0.3.

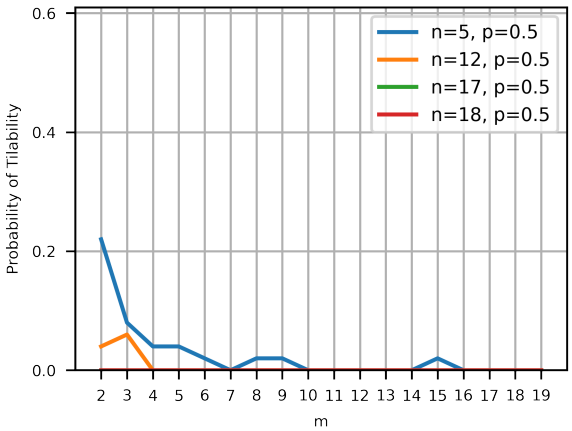


Fig. 11. Cell obstacles: Varying dimension, p=0.5.

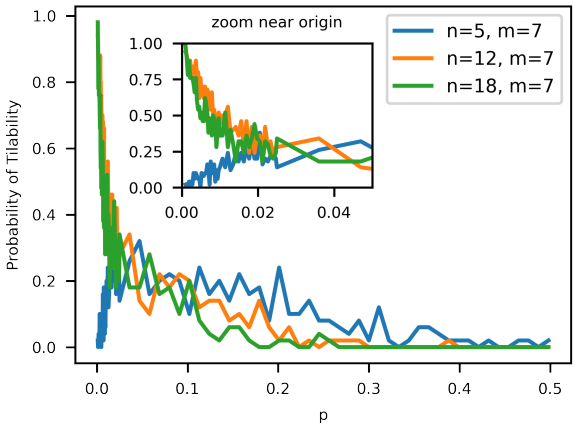


Fig. 12. Cell obstacles: Varying probability, m=7.

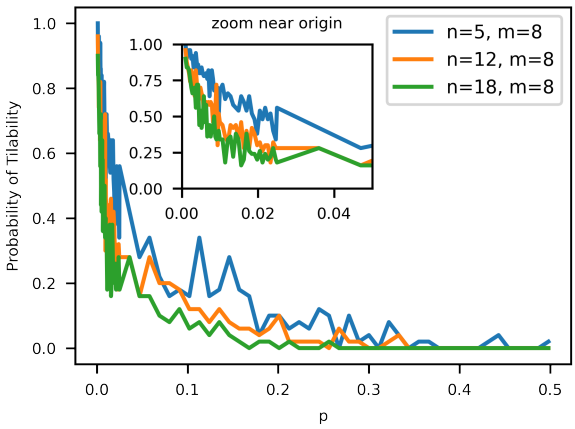


Fig. 13. Cell obstacles: Varying probability, m=8.

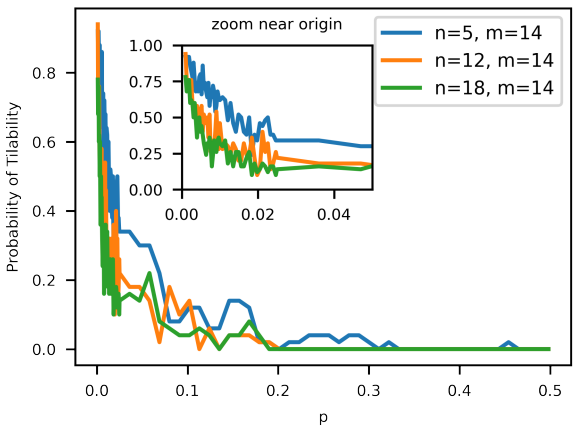


Fig. 14. Cell obstacles: Varying probability, m=14.

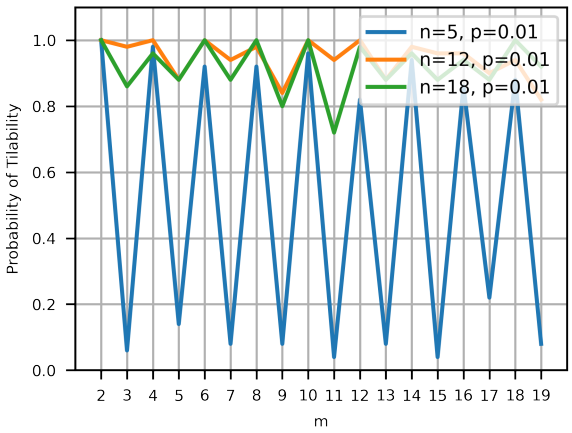


Fig. 15. Wall obstacles: Varying dimension, p=0.01.

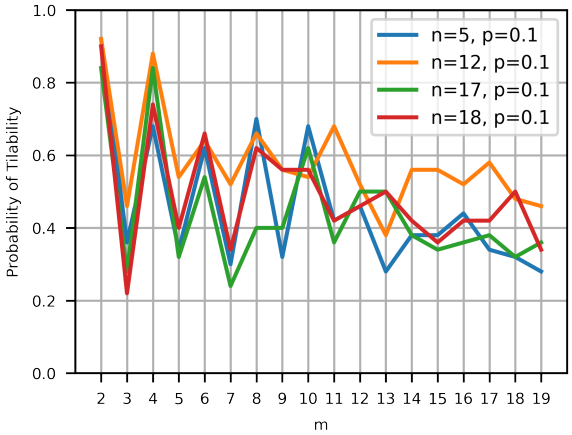


Fig. 16. Wall obstacles: Varying dimension, p=0.1.

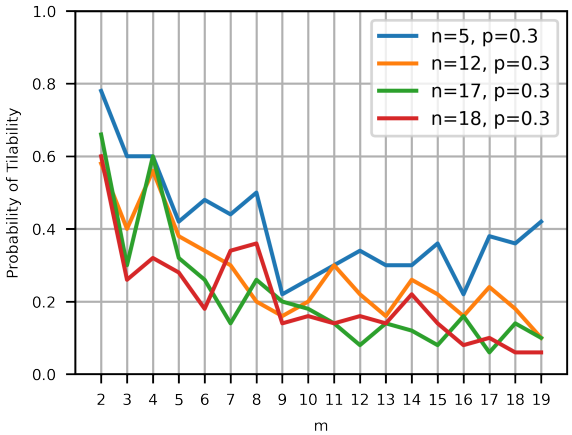


Fig. 17. Wall obstacles: Varying dimension, p=0.3.

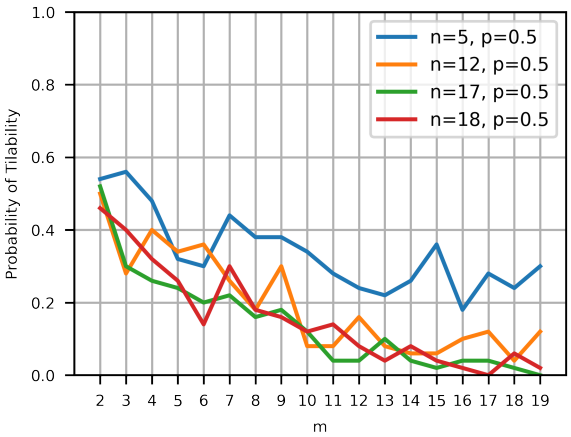


Fig. 18. Wall obstacles: Varying dimension, p=0.5.

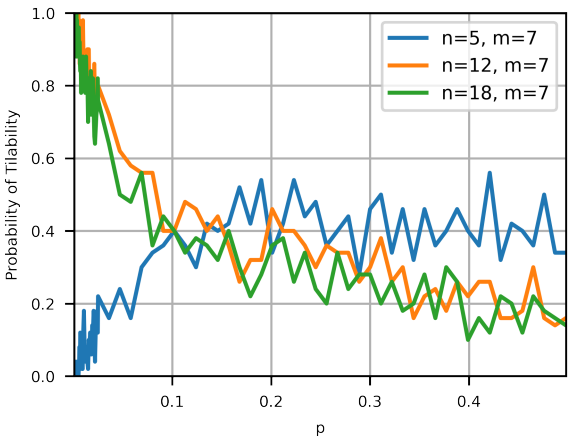


Fig. 19. Wall obstacles: Varying probability, m=7.

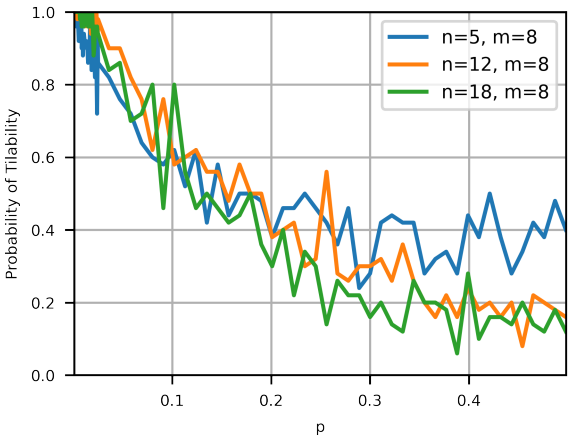


Fig. 20. Wall obstacles: Varying probability, m=8.

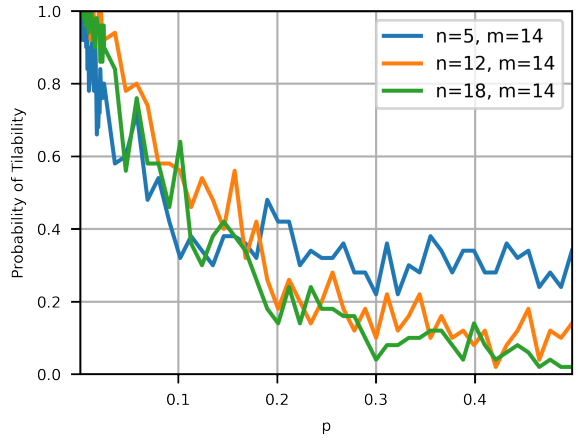


Fig. 21. Wall obstacles: Varying probability, m=14.

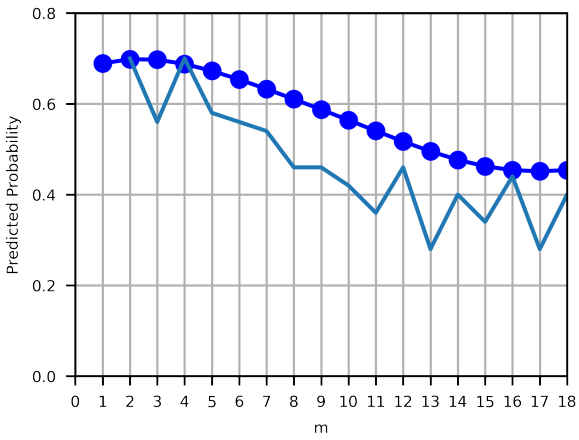


Fig. 22. Figure comparing the cell obstacles ML model predictions (dotted line) against the actual estimations for cell obstacles with p=0.01 and n=12.

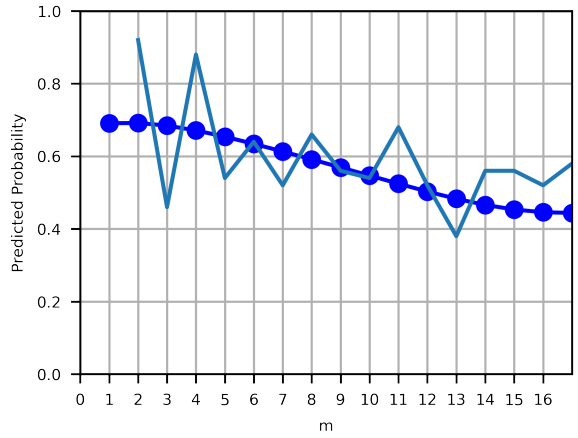


Fig. 23. Figure comparing the wall obstacles ML model predictions (dotted line) against the actual estimations for wall obstacles with p=0.1 and n=12.

VII. Conclusion

In this paper, we tackled the problem of determining the tilability of a given $n * m$ layout with obstacles using graph theory and algorithmic techniques. Our approach leveraged the Ford-Fulkerson algorithm, to efficiently solve the max-flow problem and determine tilability. By transforming the layout into a flow network, we systematically assessed the feasibility of tiling, taking into account obstacle placements.

We extended our study by incorporating a probabilistic approach to evaluate the likelihood of tilability under varying probabilities of obstacle presence. Recognizing the high computational costs associated with this analysis, we introduced machine learning, specifically Kernel Ridge Regression, to predict tilability probabilities based on layout dimensions and obstacle probabilities. This integration highlighted the potential of machine learning to enhance computational efficiency and accuracy in complex algorithmic problems.

In conclusion, our project illustrates an all-encompassing method to tackle a sophisticated tiling issue, highlighting the collaboration between conventional algorithmic approaches and contemporary machine learning strategies. This cross-disciplinary approach not only solves the current issue, but also illuminates the advantages of machine learning in addressing computationally demanding challenges.

References

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
[2] Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction (2nd ed.). Springer.
[3] Website: Wikepedia
https://www.wikipedia.org/
[4] Website: Sickit-learn
https://scikit-learn.org/stable/auto_examples/index.html
[5] Helper-tool: Chat-GPT